🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Cross-Site Scripting (XSS) reflejado en el campo de búsqueda de usuarios

ID del Ticket	**VULN-2026-012**
Activo evaluado	**Aplicación web «Búsqueda de Usuarios» (Lab Búsqueda)**
Severidad	**MEDIA** **CVSS v3.1 6.1**
Tipo	**Reflected XSS / HTML Injection — CWE-79**
Cliente	**Software Seguro — Laboratorio Búsqueda de Usuarios**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**09 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 4
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 7

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

El buscador de usuarios **muestra de vuelta en la página lo que escribe el visitante sin "neutralizarlo"**. Si en lugar de un nombre se escribe código de página web (HTML), el sitio lo **interpreta y lo ejecuta** en el navegador de quien lo ve. Se demostró inyectando contenido propio (una línea y la palabra "HACKED") que apareció renderizado en la página.

¿Por qué importa para el negocio?

Esta falla —**Cross-Site Scripting (XSS)**— permite que un atacante prepare un enlace malicioso y, cuando una víctima lo abre, ejecute **código en su navegador en el contexto del sitio**. Con eso puede **robar la sesión** de la víctima, suplantarla, realizar acciones en su nombre, capturar lo que escribe (p. ej. credenciales) o alterar visualmente la página para engañarla (phishing). El daño recae sobre los usuarios del sitio y sobre la confianza en la plataforma.

¿Qué tan grave es?

Severidad MEDIA — CVSS 6.1. La explotación es sencilla, aunque normalmente requiere que la víctima abra un enlace preparado por el atacante. Durante la prueba, el contenido inyectado (`<hr><h1>HACKED</h1>`) se renderizó en la respuesta, confirmando la falla y obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

Aplicar **codificación de salida** (escapar el HTML) sobre todo dato del usuario que se muestre en la página, de modo que el texto se vea como texto y no se interprete como código. Reforzar con una política de seguridad de contenido (CSP) y cookies protegidas.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Ejecución de código en el navegador de la víctima (robo de sesión, phishing) por XSS reflejado	Media	Medio	Alta

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El campo de búsqueda de usuarios refleja el término ingresado dentro del HTML de la respuesta **sin codificación de salida (output encoding)** ni saneamiento. En consecuencia, cualquier marcado inyectado se interpreta como parte del documento. Esto constituye un **Cross-Site Scripting (XSS) reflejado** (CWE-79).

La prueba de concepto empleó marcado HTML benigno (`<hr><h1>HACKED</h1>`), que el backend devolvió renderizado —confirmando la **inyección de HTML**—. Dado que el mismo punto no distingue entre etiquetas de presentación y etiquetas activas, la falla es **escalable a ejecución de JavaScript** (p. ej. `<script>` o manejadores de eventos como `onerror`), con el impacto completo de un XSS.

2.2 Flujo del ataque

- El atacante identifica que el campo de búsqueda refleja su entrada en la página sin escaparla.
- Construye un payload con marcado HTML/JavaScript y lo coloca en el parámetro de búsqueda (o en un enlace preparado).
- La víctima abre el enlace; el navegador interpreta el payload en el contexto del sitio.
- El código ejecutado puede robar cookies/sesión, actuar en nombre de la víctima o manipular la página (phishing).

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-79 — Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') · CWE-116 — Improper Encoding or Escaping of Output
Subtipo	Reflected XSS (demostrado como inyección de HTML; escalable a ejecución de script)
OWASP Top 10 (2021)	A03:2021 — Injection (incluye Cross-Site Scripting)
OWASP WSTG	WSTG-INPV-01 — Testing for Reflected Cross Site Scripting
CVE	No aplica (defecto propio de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Búsqueda de Usuarios» (Lab Búsqueda)
Host	`chl-<instancia>-busqueda-usuarios.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Punto vulnerable	Campo/parámetro de búsqueda de usuarios (reflejado en el HTML de la respuesta)
Parámetro	Término de búsqueda (p. ej. `q` / `buscar`)
Impacto sobre	Navegador y sesión de los usuarios que abran un enlace preparado
Alcance	Ejecución en el contexto del sitio: sesión, datos y acciones de la víctima

2.5 Causa raíz

La aplicación **inserta datos controlados por el usuario directamente en el HTML** sin aplicar codificación contextual de salida. No distingue entre texto a mostrar y marcado a interpretar, por lo que el navegador ejecuta cualquier etiqueta inyectada. La ausencia de controles de defensa en profundidad (CSP, atributos de cookie seguros) amplifica el impacto potencial.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1. Vector canónico de XSS reflejado.

CVSS Base Score	Severidad	Vector
6.1	MEDIA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: inyectar marcado en el campo de búsqueda.
Privileges Required (PR)	**None (N)**	No se requiere autenticación para construir el payload/enlace.
User Interaction (UI)	**Required (R)**	La víctima debe abrir el enlace o solicitud preparada (propio del XSS reflejado).
Scope (S)	**Changed (C)**	El código se ejecuta en el navegador de la víctima, afectando un ámbito distinto al del servidor.
Confidentiality (C)	**Low (L)**	Posible robo de datos de sesión/cookies accesibles por script en el contexto del sitio.
Integrity (I)	**Low (L)**	Posible alteración de la página o acciones en nombre de la víctima.
Availability (A)	**None (N)**	No afecta la disponibilidad del servicio.

Nota de criterio: 6.1 es la valoración estándar de un XSS reflejado. El impacto efectivo depende del contexto: si las cookies de sesión no son `HttpOnly` (robo directo de sesión) o si el XSS permite acciones sensibles, la severidad práctica es mayor. Un XSS almacenado (persistente) sobre el mismo defecto, al no requerir enlace y afectar a todos los visitantes, se valoraría más alto.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con un navegador puede replicar el hallazgo.

4.1 Requisitos previos

- Acceso a la pantalla de búsqueda de usuarios del laboratorio.
- Un navegador web (o `curl` para ver la respuesta cruda).

4.2 Pasos de reproducción

Paso 1 — Confirmar el reflejo. Buscar un texto marcador y verificar que aparece tal cual en el HTML de la respuesta.

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-busqueda-usuarios.softwareseguro.com.ar"

curl -s "$BASE/?q=marcador123" | grep marcador123
# -> el término aparece reflejado en el HTML (candidato a XSS)
```

Evidencia 1 — El término de búsqueda se refleja en la respuesta.

Paso 2 — Inyección de HTML (PoC del laboratorio). Introducir marcado en el campo de búsqueda; el backend lo devuelve renderizado.

```
# Payload requerido por el laboratorio: una línea horizontal + la palabra HACKED
<hr><h1>HACKED</h1>

# Equivalente por URL:
curl -s "$BASE/?q=%3Chr%3E%3Ch1%3EHACKED%3C%2Fh1%3E"
```

Evidencia 2 — El marcado se interpreta: aparecen la línea horizontal y el encabezado "HACKED". Inyección de HTML confirmada.

Paso 3 — Verificación del impacto (objetivo del laboratorio). La respuesta contiene el HTML renderizado solicitado y se obtiene el código del laboratorio.

```
# Código de verificación obtenido:
#   a24fc443b7c617783d96417a4f9929dc
```

Paso 4 — Escalamiento a ejecución de script (demostración controlada). El mismo punto permite ejecutar JavaScript, lo que evidencia el impacto real de un XSS.

```
# Prueba inocua de ejecución (no destructiva):
<script>alert(document.domain)</script>
<img src=x onerror=alert(document.domain)>
# En un ataque real se usaría para exfiltrar document.cookie o realizar acciones como la víctima
```

Evidencia 3 — El punto reflejado ejecuta JavaScript: XSS reflejado confirmado (más allá de la inyección de HTML).

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El término de búsqueda se muestra como texto (HTML escapado): `<hr>` se ve literalmente, sin renderizarse.	El marcado inyectado se interpreta y ejecuta en el navegador (HTML y JavaScript).

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con payloads inocuos (`<hr>`, `alert()`). No se exfiltraron cookies ni se realizaron acciones sobre terceros.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Codificación de salida contextual (output encoding). Todo dato del usuario que se muestre en la página debe escaparse según el contexto (HTML, atributo, JavaScript, URL). En la mayoría de los frameworks, usar el autoescapado del motor de plantillas y no deshabilitarlo.

```
# VULNERABLE: inserta el término sin escapar
return f"<p>Resultados para: {q}</p>"          # <- q se interpreta como HTML

# CORREGIDO: escapar la salida (o usar el autoescapado de la plantilla)
from markupsafe import escape
return f"<p>Resultados para: {escape(q)}</p>"    # <hr> se muestra como texto

# En plantillas (Jinja2/Django) el autoescapado hace esto por defecto:
# <p>Resultados para: {{ q }}</p>   -> seguro mientras no se use |safe
```

5.2 Defensa en profundidad

- **Content Security Policy (CSP)** restrictiva para limitar la ejecución de scripts inline y de orígenes no autorizados.
- **Cookies con** `HttpOnly` (inaccesibles a JavaScript) y `Secure` / `SameSite`.
- **Validación de entrada** (allow-list) del término de búsqueda como control adicional (no sustituye al escapado).
- **Sanitización con librería robusta** (p. ej. DOMPurify) solo si se requiere permitir HTML enriquecido.
- **Cabeceras de seguridad** (`X-Content-Type-Options`, etc.) y revisión de todos los puntos que reflejan entrada.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Escapar la salida del término de búsqueda (y revisar otros reflejos)	Equipo de desarrollo	Alta	Inmediato	Pendiente
Implementar CSP y marcar cookies como HttpOnly/ Secure/SameSite	Desarrollo / Infraestructura	Alta	7–15 días	Pendiente
Revisión del código por XSS (reflejado, almacenado y DOM)	Desarrollo / SecOps	Media	15–30 días	Pendiente
Test de regresión (payloads XSS = escapados) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Repetir el paso 2 con `<hr><h1>HACKED</h1>` y con `<script>` / `onerror`. El criterio de aceptación es que el marcado se muestre como texto literal (escapado) y no se ejecute, y que la CSP bloquee scripts no autorizados. Confirmar que la búsqueda legítima sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la entrada se refleja escapada y no es posible ejecutar HTML/JavaScript, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- CWE-79 — Cross-site Scripting: https://cwe.mitre.org/data/definitions/79.html
- CWE-116 — Improper Encoding or Escaping of Output: https://cwe.mitre.org/data/definitions/116.html
- OWASP — Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP — Content Security Policy Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- PortSwigger Web Security Academy — Cross-site scripting: https://portswigger.net/web-security/cross-site-scripting
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-09	J. M. F.	Creación del reporte: XSS reflejado / inyección de HTML en el campo de búsqueda de usuarios.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.